OOSE Batch 2

B.E / B.Tech. PRACTICAL END SEMESTER EXAMINATIONS

Sixth Semester

CCS356 - OBJECT ORIENTED SOFTWARE ENGINEERING LABORATORY

(Regulations 2021) - SET 2 (SIMPLIFIED)

Time: 3 Hours | **Max. Marks:** 100

MARK ALLOCATION TABLE

AIM	PROBLEM STATEMENT	UML DIAGRAMS	IMPLEMENTATION	OUTPUT	VIVA	TOTAL
10	10	30	30	10	10	**100**

General Instructions for UML Diagrams (For all Experiments)

Draw.io Link: Click here to open Draw.io (app.diagrams.net)
How to easily draw: 1. Open Draw.io -> Select "UML" from the left shape panel.
2. **Use Case:** Stick figures for Actors, Ovals for processes (e.g., Login, Register).
3. **Class Diagram:** 3-tier boxes (Top: Class Name, Middle: Attributes, Bottom: Methods).
4. **Sequence Diagram:** Vertical lifelines for objects, horizontal arrows for messages.

Ex 1: Passport Automation System

Aim: To design and implement a Passport Automation System.
Problem Statement: The system must handle applicant registration, document verification by police, and passport issuance while improving maintainability through design patterns.
UML Diagrams: Open Draw.io

- **Actors:** Applicant, Police, Regional Passport Officer (RPO).
- **Use Cases:** Apply Passport, Verify Details, Issue Passport.
- **Classes:** `Applicant`, `Verification`, `PassportSystem`.
  Implementation (Java):

```
class Applicant {
    String name;
    public Applicant(String n) { this.name = n; }
}
class Verification {
    public boolean verify(Applicant a) {
        System.out.println("Police Verification completed for: " +
a.name);
        return true;
    }
}
public class Main {
    public static void main(String[] args) {
        Applicant app = new Applicant("Ravi");
        Verification v = new Verification();
        if(v.verify(app)) {
            System.out.println("Passport Issued Successfully to " +
app.name);
        }
    }
}
```

Output:

```
Police Verification completed for: Ravi
Passport Issued Successfully to Ravi
```

Result: The Passport Automation System was successfully designed and implemented.

Ex 2: Book Bank System

Aim: To design and implement a Book Bank System.
Problem Statement: The system must allow users to search, borrow, and return books while managing the domain and technical service layers.
UML Diagrams: Open Draw.io

- **Actors:** Student, Librarian.
- **Use Cases:** Search Book, Issue Book, Return Book.
- **Classes:** `Student`, `Book`, `Library`.
  Implementation (Java):

```
class Book {
    String title; int copies;
    public Book(String t, int c) { this.title = t; this.copies = c; }
```

```
    public void issue() {
        if(copies > 0) {
            copies--;
            System.out.println(title + " issued. Copies left: " + copies);
        } else {
            System.out.println("Out of stock.");
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Book b = new Book("Design Patterns", 2);
        b.issue();
    }
}
```

Output:

```
Design Patterns issued. Copies left: 1
```

Result: The Book Bank System was successfully designed and implemented.

Ex 3: Exam Registration System

Aim: To design and implement an Exam Registration System.
Problem Statement: The system must handle student exam course selection, fee payment, and hall ticket generation.
UML Diagrams: Open Draw.io

- **Actors:** Student, Admin.
- **Use Cases:** Register Exam, Pay Fees, Generate Hall Ticket.
- **Classes:** `Student`, `Exam`, `Registration`.
  Implementation (Java):

```
class Registration {
    String studentName, subject;
    public void register(String name, String sub) {
        this.studentName = name;
        this.subject = sub;
        System.out.println("Exam Registration Confirmed.");
        System.out.println("Name: " + name + " | Subject: " + sub);
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        Registration reg = new Registration();
        reg.register("Alice", "Software Engineering");
    }
}
```

Output:

```
Exam Registration Confirmed.
Name: Alice | Subject: Software Engineering
```

Result: The Exam Registration System was successfully designed and implemented.

Ex 4: Stock Maintenance System

Aim: To design and implement a Stock Maintenance System.
Problem Statement: The system must manage inventory, track items, and update stock levels across different software layers.
UML Diagrams: Open Draw.io

- **Actors:** Store Manager, Supplier.
- **Use Cases:** Add Item, Update Stock, Generate Report.
- **Classes:** `Admin`, `Item`, `Inventory`.
  Implementation (Java):

```
class Item {
    String itemName;
    int quantity;
    public void updateStock(int addedQty) {
        quantity += addedQty;
        System.out.println(itemName + " stock updated. Total: " +
quantity);
    }
}
public class Main {
    public static void main(String[] args) {
        Item item = new Item();
        item.itemName = "Monitors";
        item.quantity = 15;
        item.updateStock(5);
    }
}
```

Output:

```
Monitors stock updated. Total: 20
```

Result: The Stock Maintenance System was successfully designed and implemented.

Ex 5: Online Course Registration System

Aim: To design and implement an Online Course Registration System.
Problem Statement: The system must allow users to browse courses, enroll, and process payments while demonstrating appropriate design patterns.
UML Diagrams: Open Draw.io

- **Actors:** User, Instructor.
- **Use Cases:** View Courses, Enroll, Make Payment.
- **Classes:** `User`, `Course`, `Payment`.
  Implementation (Java):

```
class Course {
    String cName;
    public Course(String name) { this.cName = name; }
}
class User {
    public void enroll(Course c) {
        System.out.println("Successfully enrolled in the online course: "
+ c.cName);
    }
}
public class Main {
    public static void main(String[] args) {
        User u = new User();
        u.enroll(new Course("Java Programming Masterclass"));
    }
}
```

Output:

```
Successfully enrolled in the online course: Java Programming Masterclass
```

Result: The Online Course Registration System was successfully designed and implemented.

Ex 6: Airline/Airway Reservation System

Aim: To design and implement an Airline Reservation System.
Problem Statement: The system must handle flight searches, seat reservations, and ticket cancellations across the domain and UI layers.
UML Diagrams: Open Draw.io

- **Actors:** Passenger, Ticket Agent.
- **Use Cases:** Search Flight, Book Ticket, Cancel Ticket.
- **Classes:** `Passenger`, `Flight`, `Ticket`.
  Implementation (Java):

```
class Flight {
    String flightNo; int availableSeats;
    public Flight(String fNo, int seats) { this.flightNo = fNo;
this.availableSeats = seats; }
    public void bookSeat() {
        if(availableSeats > 0) {
            availableSeats--;
            System.out.println("Ticket Booked on " + flightNo + ". Seats
remaining: " + availableSeats);
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Flight f = new Flight("AI-202", 50);
        f.bookSeat();
    }
}
```

Output:

```
Ticket Booked on AI-202. Seats remaining: 49
```

Result: The Airline Reservation System was successfully designed and implemented.

Ex 7: Software Personnel Management System

Aim: To design and implement a Software Personnel Management System.
Problem Statement: The system must track employee records, manage departments, and update software personnel data.
UML Diagrams: Open Draw.io

- **Actors:** HR Admin, Employee.
- **Use Cases:** Add Employee, Assign Project, Update Details.

- **Classes:** `Employee`, `Project`, `HRManager`.
  Implementation (Java):

```
class Employee {
    int empId; String name;
    public void displayDetails() {
        System.out.println("Emp ID: " + empId + " | Name: " + name);
    }
}
public class Main {
    public static void main(String[] args) {
        Employee e = new Employee();
        e.empId = 1045;
        e.name = "John Doe";
        System.out.println("Personnel Record Created:");
        e.displayDetails();
    }
}
```

Output:

```
Personnel Record Created:
Emp ID: 1045 | Name: John Doe
```

Result: The Software Personnel Management System was successfully designed and implemented.

Ex 8: Credit Card Processing System

Aim: To design and implement a Credit Card Processing System.
Problem Statement: The system must authenticate card details, process transactions, and check credit limits.
UML Diagrams: Open Draw.io

- **Actors:** Cardholder, Merchant, Bank.
- **Use Cases:** Swipe Card, Authenticate, Process Payment.
- **Classes:** `CreditCard`, `Transaction`, `BankServer`.
  Implementation (Java):

```
class CreditCard {
    double limit = 50000.0;
    public void swipe(double amount) {
        if(amount <= limit) {
            limit -= amount;
```

```
            System.out.println("Payment Approved. Amount: $" + amount);
            System.out.println("Available Limit: $" + limit);
        } else {
            System.out.println("Declined: Over Limit.");
        }
    }
}
public class Main {
    public static void main(String[] args) {
        CreditCard cc = new CreditCard();
        cc.swipe(12000.0);
    }
}
```

Output:

```
Payment Approved. Amount: $12000.0
Available Limit: $38000.0
```

Result: The Credit Card Processing System was successfully designed and implemented.

Ex 9: E-book Management System

Aim: To design and implement an E-book Management System.
Problem Statement: The system must handle digital book inventory, reader subscriptions, and downloads.
UML Diagrams: Open Draw.io

- **Actors:** Reader, Admin.
- **Use Cases:** Search E-book, Read Online, Download E-book.
- **Classes:** `Reader`, `Ebook`, `LibrarySystem`.
  Implementation (Java):

```
class Ebook {
    String title, format;
    public void download() {
        System.out.println("Downloading E-book: " + title + " [" + format
+ "]");
        System.out.println("Download Complete.");
    }
}
public class Main {
    public static void main(String[] args) {
        Ebook book = new Ebook();
        book.title = "UML Distilled";
```

```
        book.format = "PDF";
        book.download();
    }
}
```

Output:

```
Downloading E-book: UML Distilled [PDF]
Download Complete.
```

Result: The E-book Management System was successfully designed and implemented.

Ex 10: Recruitment System

Aim: To design and implement a Recruitment System.
Problem Statement: The system must allow candidates to upload resumes, apply for jobs, and let interviewers schedule interviews.
UML Diagrams: Open Draw.io

- **Actors:** Candidate, HR/Recruiter.
- **Use Cases:** Upload Resume, Apply Job, Schedule Interview.
- **Classes:** `Candidate`, `JobPost`, `Interview`.
  Implementation (Java):

```
class Candidate {
    String name;
    public void applyForJob(String jobTitle) {
        System.out.println(name + " has successfully applied for the
position: " + jobTitle);
    }
}
public class Main {
    public static void main(String[] args) {
        Candidate c = new Candidate();
        c.name = "Sarah";
        c.applyForJob("Senior Software Engineer");
    }
}
```

Output:

```
Sarah has successfully applied for the position: Senior Software Engineer
```

Result: The Recruitment System was successfully designed and implemented.

Ex 11: Online Examination System

Aim: To design and implement an Online Examination System.

Problem Statement: The system must handle student logins, test execution, auto-grading, and result generation.

UML Diagrams: Open Draw.io

- **Actors:** Student, Examiner.
- **Use Cases:** Take Test, Submit Test, View Score.
- **Classes:** `Student`, `Exam`, `Result`.

Implementation (Java):

```
class Exam {
    String subject;
    public Exam(String sub) { this.subject = sub; }

    public int evaluate(int correctAnswers) {
        return correctAnswers * 10; // 10 marks per question
    }
}

public class Main {
    public static void main(String[] args) {
        Exam e = new Exam("Java Programming");
        int score = e.evaluate(8);
        System.out.println("Exam: " + e.subject);
        System.out.println("Total Score: " + score + "/100");
    }
}
```

Output:

```
Exam: Java Programming
Total Score: 80/100
```

Result: The Online Examination System was successfully designed and implemented.

Ex 12: Conference Management Processing System

Aim: To design and implement a Conference Management System.

Problem Statement: The system must handle paper submissions, reviewer assignments, and conference registrations.

UML Diagrams: Open Draw.io

- **Actors:** Author, Reviewer, Admin.
- **Use Cases:** Submit Paper, Review Paper, Register for Conference.
- **Classes:** `Author`, `Paper`, `Conference`.

Implementation (Java):

```
class Paper {
    String title, status;
    public Paper(String t) { this.title = t; this.status = "Submitted"; }

    public void reviewPaper() {
        this.status = "Accepted";
        System.out.println("Paper '" + title + "' has been " + status);
    }
}

public class Main {
    public static void main(String[] args) {
        Paper p = new Paper("AI in Healthcare");
        System.out.println("Status: " + p.status);
        p.reviewPaper();
    }
}
```

Output:

```
Status: Submitted
Paper 'AI in Healthcare' has been Accepted
```

Result: The Conference Management System was successfully designed and implemented.

Ex 13: BPO Management System

Aim: To design and implement a BPO Management System.

Problem Statement: The system must track client calls, resolve customer issues, and manage agent performance.

UML Diagrams: Open Draw.io

- **Actors:** Customer, Agent, Manager.
- **Use Cases:** Log Call, Resolve Issue, View Performance.
- **Classes:** `Customer`, `Ticket`, `Agent`.

Implementation (Java):

```
class Ticket {
    int ticketId; String issue, status;

    public Ticket(int id, String i) { this.ticketId = id; this.issue = i;
this.status = "Open"; }

    public void resolve() {
        this.status = "Resolved";
        System.out.println("Ticket " + ticketId + " (" + issue + ") is now
" + status);
    }
}

public class Main {
    public static void main(String[] args) {
        Ticket t = new Ticket(101, "Billing Error");
        t.resolve();
    }
}
```

Output:

```
Ticket 101 (Billing Error) is now Resolved
```

Result: The BPO Management System was successfully designed and implemented.

Ex 14: Library Management Database System

Aim: To design and implement a Library Management Database System.

Problem Statement: The system must manage book inventory, issue/return processes, and calculate fines for late returns.

UML Diagrams: Open Draw.io

- **Actors:** Member, Librarian.
- **Use Cases:** Borrow Book, Return Book, Pay Fine.
- **Classes:** `Member`, `Book`, `Transaction`.

Implementation (Java):

```
class Transaction {
    String bookName;
    int lateDays;

    public void returnBook(String name, int days) {
        this.bookName = name;
        this.lateDays = days;
        System.out.println(bookName + " returned.");
        if(lateDays > 0) {
            System.out.println("Late Fine: $" + (lateDays * 2));
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Transaction t = new Transaction();
        t.returnBook("Software Engineering", 3);
    }
}
```

Output:

```
Software Engineering returned.
Late Fine: $6
```

Result: The Library Management Database System was successfully designed and implemented.

Ex 15: Student Information System

Aim: To design and implement a Student Information System.

Problem Statement: The system must track student attendance, academic grades, and personal details securely.

UML Diagrams: Open Draw.io

- **Actors:** Student, Teacher, Admin.
- **Use Cases:** Update Details, View Grades, Mark Attendance.
- **Classes:** `Student`, `AcademicRecord`, `System`.

Implementation (Java):

```
class StudentRecord {
    String name; float cgpa;

    public StudentRecord(String n, float c) { this.name = n; this.cgpa =
c; }

    public void display() {
        System.out.println("Student: " + name + " | CGPA: " + cgpa);
        if(cgpa >= 8.0) System.out.println("Status: Distinction");
    }
}

public class Main {
    public static void main(String[] args) {
        StudentRecord s = new StudentRecord("Mike", 8.5f);
        s.display();
    }
}
```

Output:

```
Student: Mike | CGPA: 8.5
Status: Distinction
```

Result: The Student Information System was successfully designed and implemented.

Ex 16: Blood Bank Information Portal

Aim: To design and implement a Blood Bank Information Portal.

Problem Statement: The system must register blood donors, track available blood groups in the inventory, and manage hospital requests.

UML Diagrams: Open Draw.io

- **Actors:** Donor, Hospital, Admin.
- **Use Cases:** Register Donor, Request Blood, Check Inventory.
- **Classes:** `Donor`, `BloodStock`, `Request`.

Implementation (Java):

```
class BloodStock {
    String group; int units;

    public BloodStock(String g, int u) { this.group = g; this.units = u; }
```

```
    public void requestBlood(int requestedUnits) {
        if(units >= requestedUnits) {
            units -= requestedUnits;
            System.out.println("Request Approved. Remaining " + group + "
units: " + units);
        } else {
            System.out.println("Insufficient Stock for " + group);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        BloodStock b = new BloodStock("O+ve", 10);
        b.requestBlood(4);
    }
}
```

Output:

```
Request Approved. Remaining O+ve units: 6
```

Result: The Blood Bank Information Portal was successfully designed and implemented.

Ex 17: E-Voting System

Aim: To design and implement an E-Voting System.

Problem Statement: The system must verify voter identity, allow them to cast votes securely, and automatically tabulate the results.

UML Diagrams: Open Draw.io

- **Actors:** Voter, Election Commission.
- **Use Cases:** Verify ID, Cast Vote, View Results.
- **Classes:** `Voter`, `Candidate`, `Election`.

Implementation (Java):

```
class Election {
    int candidateAVotes = 0;

    public void castVote(String voterId) {
        candidateAVotes++;
        System.out.println("Vote cast successfully by Voter ID: " +
```

```
voterId);
    }
    public void showResults() {
        System.out.println("Candidate A Total Votes: " + candidateAVotes);
    }
}

public class Main {
    public static void main(String[] args) {
        Election e = new Election();
        e.castVote("VOTER_9012");
        e.showResults();
    }
}
```

Output:

```
Vote cast successfully by Voter ID: VOTER_9012
Candidate A Total Votes: 1
```

Result: The E-Voting System was successfully designed and implemented.

Ex 18: Online Course Registration System

Aim: To design and implement an Online Course Registration System.

Problem Statement: The system must allow students to select elective courses, verify prerequisites, and confirm enrollment.

UML Diagrams: Open Draw.io

- **Actors:** Student, Admin.
- **Use Cases:** Select Course, Check Prerequisites, Confirm Enrollment.
- **Classes:** `Student`, `Course`, `Enrollment`.

Implementation (Java):

```
class Enrollment {
    public void enrollStudent(String name, String course) {
        System.out.println("Processing enrollment...");
        System.out.println("Student " + name + " is officially enrolled in
" + course);
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Enrollment en = new Enrollment();
        en.enrollStudent("David", "Cloud Computing");
    }
}
```

Output:

```
Processing enrollment...
Student David is officially enrolled in Cloud Computing
```

Result: The Online Course Registration System was successfully designed and implemented.

Ex 19: Internet Banking Portal

Aim: To design and implement an Internet Banking Portal.

Problem Statement: The system must handle secure user logins, inter-account fund transfers, and balance inquiries.

UML Diagrams: Open Draw.io

- **Actors:** Customer, Bank Server.
- **Use Cases:** Login, Transfer Funds, Check Balance.
- **Classes:** `Account`, `Transaction`, `Customer`.

Implementation (Java):

```
class Account {
    int accNo; double balance;

    public Account(int acc, double bal) { this.accNo = acc; this.balance =
bal; }

    public void transfer(Account target, double amount) {
        if(balance >= amount) {
            this.balance -= amount;
            target.balance += amount;
            System.out.println("Transferred $" + amount + " to Account " +
target.accNo);
            System.out.println("Your new balance: $" + this.balance);
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Account myAcc = new Account(101, 5000.0);
        Account targetAcc = new Account(102, 1000.0);
        myAcc.transfer(targetAcc, 1500.0);
    }
}
```

Output:

```
Transferred $1500.0 to Account 102
Your new balance: $3500.0
```

Result: The Internet Banking Portal was successfully designed and implemented.

Ex 20: Banking System

Aim: To design and implement a General Banking System.

Problem Statement: The system must manage customer accounts, process cash deposits, and handle cash withdrawals securely.

UML Diagrams: Open Draw.io

- **Actors:** Customer, Bank Teller.
- **Use Cases:** Deposit Money, Withdraw Money, Print Statement.
- **Classes:** `Customer`, `BankAccount`, `Bank`.

Implementation (Java):

```
class BankAccount {
    String owner; double balance;

    public BankAccount(String o, double b) { this.owner = o; this.balance
= b; }

    public void deposit(double amount) {
        balance += amount;
        System.out.println(owner + " deposited $" + amount + ". Balance:
$" + balance);
    }

    public void withdraw(double amount) {
        if(balance >= amount) {
            balance -= amount;
```

```
            System.out.println(owner + " withdrew $" + amount + ".
Balance: $" + balance);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Emma", 2000.0);
        acc.deposit(500.0);
        acc.withdraw(300.0);
    }
}
```

Output:

```
Emma deposited $500.0. Balance: $2500.0
Emma withdrew $300.0. Balance: $2200.0
```

Result: The Banking System was successfully designed and implemented.

Ex 21: ATM System

Aim: To design and implement an ATM System.

Problem Statement: The system must handle secure user PIN verification, cash withdrawals, and balance inquiries.

UML Diagrams: Open Draw.io

- **Actors:** Customer, Bank Server.
- **Use Cases:** Verify PIN, Withdraw Cash, Check Balance.
- **Classes:** `ATM`, `Account`, `Customer`.

Implementation (Java):

```
class Account {
    int pin = 1234;
    double balance = 5000.0;

    public void withdraw(int enteredPin, double amount) {
        if (enteredPin == pin) {
            if (balance >= amount) {
                balance -= amount;
                System.out.println("Dispensing Cash: $" + amount);
                System.out.println("Remaining Balance: $" + balance);
```

```
            } else {
                System.out.println("Insufficient Funds!");
            }
        } else {
            System.out.println("Invalid PIN!");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Account acc = new Account();
        acc.withdraw(1234, 1500.0);
    }
}
```

Output:

```
Dispensing Cash: $1500.0
Remaining Balance: $3500.0
```

Result: The ATM System was successfully designed and implemented.

Ex 22: Hospital Management System

Aim: To design and implement a Hospital Management System.

Problem Statement: The system must manage patient registration, doctor appointments, and medical billing.

UML Diagrams: Open Draw.io

- **Actors:** Patient, Receptionist, Doctor.
- **Use Cases:** Register Patient, Book Appointment, Generate Bill.
- **Classes:** `Patient`, `Doctor`, `Appointment`.

Implementation (Java):

```
class Doctor {
    String name, specialization;
    public Doctor(String n, String s) { this.name = n; this.specialization
= s; }
}

class Appointment {
    public void book(String patientName, Doctor doc) {
```

```
        System.out.println("Appointment Confirmed.");
        System.out.println("Patient: " + patientName + " | Doctor: " +
doc.name + " (" + doc.specialization + ")");
    }
}

public class Main {
    public static void main(String[] args) {
        Doctor doc = new Doctor("Dr. Smith", "Cardiologist");
        Appointment app = new Appointment();
        app.book("James", doc);
    }
}
```

Output:

```
Appointment Confirmed.
Patient: James | Doctor: Dr. Smith (Cardiologist)
```

Result: The Hospital Management System was successfully designed and implemented.

Ex 23: Foreign Trading System

Aim: To design and implement a Foreign Trading System.

Problem Statement: The system must handle import/export details, trade transactions across borders, and currency conversion.

UML Diagrams: Open Draw.io

- **Actors:** Trader, Customs Agent, Bank.
- **Use Cases:** Register Trade, Convert Currency, Process Export.
- **Classes:** `Trader`, `TradeTransaction`, `CurrencyConverter`.

Implementation (Java):

```
class CurrencyConverter {
    public double convertUsdToInr(double usdAmount) {
        return usdAmount * 83.0; // Assume 1 USD = 83 INR
    }
}

class TradeTransaction {
    public void processExport(String item, double usdValue) {
        CurrencyConverter converter = new CurrencyConverter();
        double inrValue = converter.convertUsdToInr(usdValue);
```

```
        System.out.println("Exporting Item: " + item);
        System.out.println("Transaction Value: $" + usdValue + " (INR " +
inrValue + ")");
    }
}

public class Main {
    public static void main(String[] args) {
        TradeTransaction trade = new TradeTransaction();
        trade.processExport("Automobile Parts", 5000.0);
    }
}
```

Output:

```
Exporting Item: Automobile Parts
Transaction Value: $5000.0 (INR 415000.0)
```

Result: The Foreign Trading System was successfully designed and implemented.

Ex 24: Payroll Processing System

Aim: To design and implement a Payroll Processing System.

Problem Statement: The system must calculate employee salaries, manage tax deductions, and generate monthly payslips.

UML Diagrams: Open Draw.io

- **Actors:** HR Admin, Employee.
- **Use Cases:** Calculate Salary, Deduct Taxes, Generate Payslip.
- **Classes:** `Employee`, `Payroll`, `Payslip`.

Implementation (Java):

```
class Payroll {
    public void generatePayslip(String empName, double baseSalary, double
bonus) {
        double tax = baseSalary * 0.10; // 10% tax deduction
        double netSalary = (baseSalary + bonus) - tax;

        System.out.println("--- PAYSLIP for " + empName + " ---");
        System.out.println("Base Salary: $" + baseSalary);
        System.out.println("Bonus: $" + bonus);
        System.out.println("Tax Deduction: -$" + tax);
        System.out.println("Net Salary: $" + netSalary);
```

```
    }
}

public class Main {
    public static void main(String[] args) {
        Payroll p = new Payroll();
        p.generatePayslip("Alice Johnson", 6000.0, 500.0);
    }
}
```

Output:

```
--- PAYSLIP for Alice Johnson ---
Base Salary: $6000.0
Bonus: $500.0
Tax Deduction: -$600.0
Net Salary: $5900.0
```

Result: The Payroll Processing System was successfully designed and implemented.

Ex 25: University Database System

Aim: To design and implement a University Database System.

Problem Statement: The system must maintain student and faculty records, department details, and course allocations.

UML Diagrams: Open Draw.io

- **Actors:** Admin, Student, Faculty.
- **Use Cases:** Add Student, Assign Faculty, View Department.
- **Classes:** `University`, `Department`, `Student`.

Implementation (Java):

```
class Department {
    String deptName;
    public Department(String name) { this.deptName = name; }
}

class Student {
    String name;
    public void enroll(Department dept) {
        System.out.println("Student '" + name + "' successfully enrolled
in the " + dept.deptName + " department.");
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Department cs = new Department("Computer Science");
        Student s = new Student();
        s.name = "Mark";
        s.enroll(cs);
    }
}
```

Output:

```
Student 'Mark' successfully enrolled in the Computer Science department.
```

Result: The University Database System was successfully designed and implemented.
